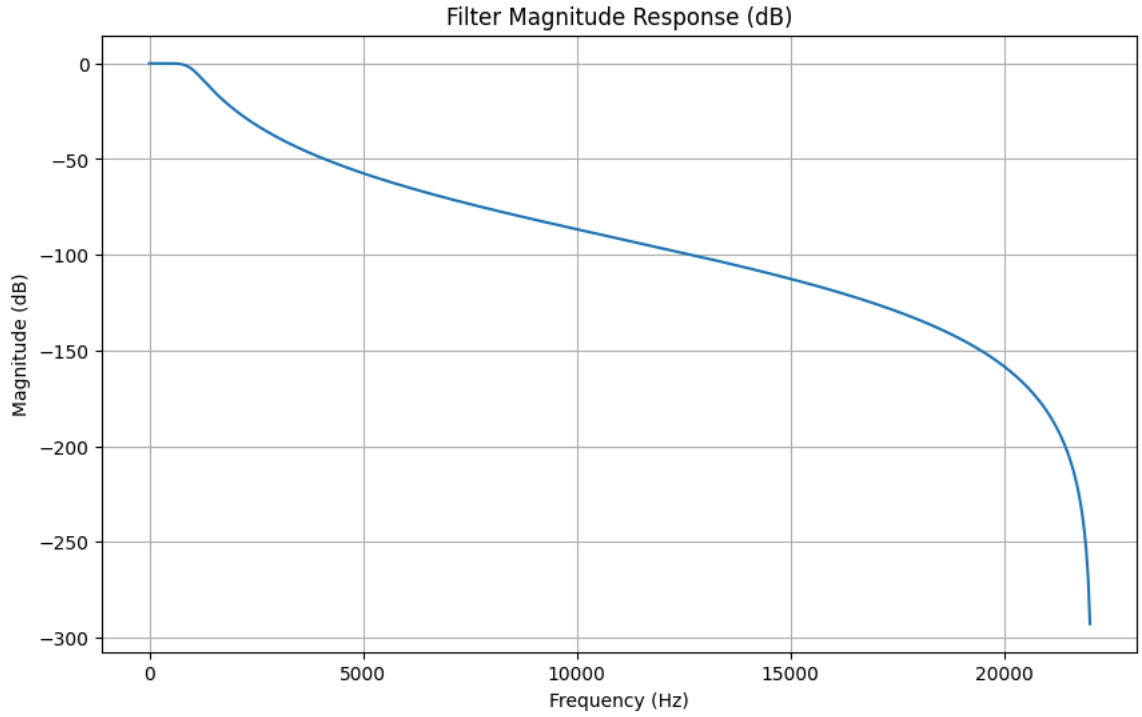
```
import matplotlib.pyplot as plt
import numpy as np
from scipy.signal import butter, cheby1, freqz, lfilter
import librosa
import soundfile as sf
import requests


def download_song(url, filename="input_song.mp3"):
    """Download MP3 from the given URL"""
    response = requests.get(url)
    with open(filename, "wb") as f:
        f.write(response.content)
    print(f"Downloaded song: {filename}")
    return filename


def design_butterworth_filter(filter_order, cutoff_frequency, sampling_frequency):
    nyquist_frequency = sampling_frequency / 2
    normalized_cutoff = cutoff_frequency / nyquist_frequency
    digital_b, digital_a = butter(
        filter_order, normalized_cutoff, btype="low", analog=False
    )
    return digital_b, digital_a


def design_chebyshev_filter(filter_order, cutoff_frequency, sampling_frequency, ripple):
    nyquist_frequency = sampling_frequency / 2
    normalized_cutoff = cutoff_frequency / nyquist_frequency
    digital_b, digital_a = cheby1(
        filter_order, ripple, normalized_cutoff, btype="low", analog=False
    )
    return digital_b, digital_a


def plot_filter_response(digital_b, digital_a, sampling_frequency):
    frequency, magnitude_response = freqz(digital_b, digital_a, fs=sampling_frequency)

    plt.figure(figsize=(10, 6))
    plt.plot(frequency, 20 * np.log10(np.abs(magnitude_response)))
    plt.title("Filter Magnitude Response (dB)")
    plt.xlabel("Frequency (Hz)")
    plt.ylabel("Magnitude (dB)")
    plt.grid(True)
    plt.show()


def apply_filter_to_audio(input_file, output_file, filter_b, filter_a):
    audio_data, sr = librosa.load(input_file, sr=None)  # load MP3
    filtered_audio = lfilter(filter_b, filter_a, audio_data)
    sf.write(output_file, filtered_audio, sr)
    print(f"Filtered audio saved as: {output_file}")


    # Plot waveforms
    plt.figure(figsize=(12, 6))
    plt.subplot(2, 1, 1)
    plt.plot(audio_data)
    plt.title("Original Audio Waveform")
    plt.xlabel("Samples")
    plt.ylabel("Amplitude")

    plt.subplot(2, 1, 2)
    plt.plot(filtered_audio)
    plt.title("Filtered Audio Waveform (Low-Pass)")
    plt.xlabel("Samples")
    plt.ylabel("Amplitude")
    plt.tight_layout()
    plt.show()


filter_order = 4
cutoff_frequency = 1000
ripple = 0.5

song_url = "https://www.soundhelix.com/examples/mp3/SoundHelix-Song-1.mp3"

input_mp3 = download_song(song_url)

y, sr = librosa.load(input_mp3, sr=None)
```

```
b_butter, a_butter = design_butterworth_filter(filter_order, cutoff_frequency, sr)
plot_filter_response(b_butter, a_butter, sr)
apply_filter_to_audio(input_mp3, "butter_filtered.wav", b_butter, a_butter)

b_cheby, a_cheby = design_chebyshev_filter(filter_order, cutoff_frequency, sr, ripple)
plot_filter_response(b_cheby, a_cheby, sr)
apply_filter_to_audio(input_mp3, "cheby_filtered.wav", b_cheby, a_cheby)
```

```
Downloaded song: input_song.mp3
```




```
Filtered audio saved as: butter_filtered.wav
```